# AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

## Faculty of Engineering

## Lab Report

### Experiment # 06

### Experiment Title: Communication between two Arduino Boards using SPI.

| | | | |
|---|---|---|---|
| **Date of Performance:** | 14/12/2025 | **Date of Submission:** | 20/12/2025 |
| **Course Title:** | Microprocessor and Embedded Systems Lab | | |
| **Course Code:** | EEE4103 | **Section:** | **O** |
| **Semester:** | Fall 2024-25 | **Degree Program:** | BSc in CSE/EEE |
| **Course Teacher:** | **Dr.Md.Rukonuzzaman** | | |



**Group # 03**


| Sl No | Name | ID | PROGRAM | SIGNATURE |
|---|---|---|---|---|
| 1 | **Nafiur Rahman Siam** | **23-50834-1** | **BSc in CSE** | |
| 2 | **Arfanul Hoque Nehal** | **23-54270-3** | **BSc in CSE** | |
| 3 | **Mashkur Ibtesham Wazed** | **23-54266-3** | **BSc in CSE** | |
| 4 | **Chinmoy Guha** | **22-48056-2** | **BSc in CSE** | |
| 5 | **Suvra Chakraborty** | **22-48067-2** | **BSc in CSE** | |
| 6 | **MOHAMMAD ANSAR UDDIN** | **22-47975-2** | **BSc in CSE** | |

| *Faculty use only* | | |
|---|---|---|
| FACULTY COMMENTS | **Marks Obtained** | |
| | **Total Marks** | |

# Table of Contents



## Marking Rubrics (to be filled by Faculty):

| Level Category | Excellent [5] | Proficient [4] | Good [3] | Acceptable [2] | Unacceptable [1] | No Response [0] |
|---|---|---|---|---|---|---|
| **Title and Objectives** | Able to clarify the understanding of the lab, no issues are missing, and formatting is good. | Able to clarify the understanding of the lab experiment, no issues are missing, but its formatting is not good. | Able to clarify the understanding of the lab experiment, but a few wrong issues, and its formatting is not good. | Able to clarify the understanding of the lab experiment, but it lacks a few important issues of the experiment, without maintaining the format. | Unable to clarify the understanding of the lab experiment. | No Response/ copied from others/ identical submissions with gross errors/image file printed |
| **Codes and Methods** | Able to explain the experimental codes and simulation methods using Proteus very well. | Able to explain the experimental codes and simulation methods using Proteus, but it is not formatted well. | Able to explain the experimental codes, but the simulation method using Proteus is not explained well. | Presents the experimental code but doesn't explain the simulation methods using Proteus. | Presents the experimental code but doesn't explain the simulation methods using Proteus. | |
| **Results** | Key results and images are there. Figures/Tables have all identifications and refer to them properly in the text. | Key results and images are there. Figures/Tables have all identifications, such as the axis labels, numbers, and captions, with a few minor errors; the texts refer to them. | Key results and images are there. Figures/Tables lack a few identifications, such as the axis labels, numbers, and captions; the texts refer to them. | Misses several key results and images. Figures/Tables lack identification, such as the axis labels, numbers, and captions; the texts don't refer to them. | Major results, such as experimental and simulation results' images, are not included. Figures and tables are poorly constructed or not presented. | |

| | | | | | | |
|---|---|---|---|---|---|---|
| **Discussion and Conclusion** | Proper interpretation of results and summarizes the results to conclude, discusses its applications in real-life situations to connect with the report's conclusion. | Proper interpretation of results and summarizes the results to conclude, but didn't discuss its applications in real-life situations to connect with the conclusion of the report. | The interpretation of results is presented. However, there is a disconnect between the results and the discussion. | Misses the interpretation of key results. There is little connection between the results and the discussion. | Very poor interpretation of the results. No connection between results and discussions. | |
| **Question and Answer** | Able to produce all questions' answers correctly, maintaining the lab report format. | Able to produce all questions' answers but didn't maintain the lab report format. | Able to produce all questions' answers but wrong answers to a few questions. | Able to produce all questions' answers, but wrong/missing answers to multiple questions. | Unable to produce all the questions' answers, and completely wrong answers. | |
| **Comments** | | | | | | **Total Marks (25)** |

## Objectives:

The objectives of this experiment are to

1. Study the SPI protocol used in Arduino.
2. Write assembly language programming code for SPI communication with Arduinos.
3. Use SPI protocol for communication between two Arduinos.
4. Build a circuit to control the master side LED by the push button at the slave side and vice versa using the SPI Serial communication protocol.
5. Know the working principles of the SPI used in Arduino.

## Equipment List:

1. Arduino IDE (2.0.1 or any recent version)
2. Arduino UNO (2)
3. LED (2)
4. Push Button (2)
5. Resistors 10 k, 2.2 k (2 + 2)
6. Breadboard
7. Connecting Wires

## Circuit Diagram:

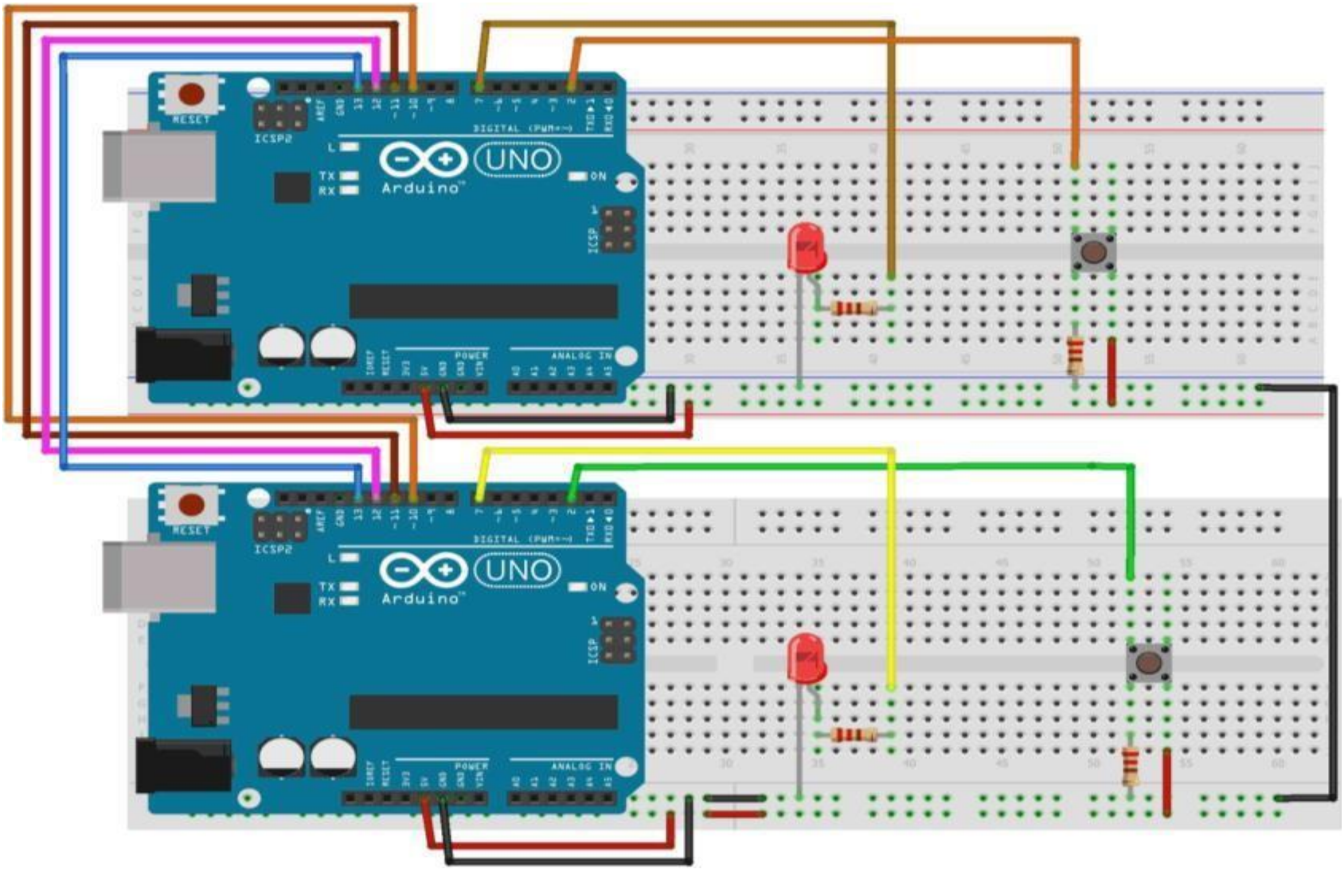

**Figure 1:** Circuit Diagram using two LED and two push buttons.

**Experimental Output Results:**

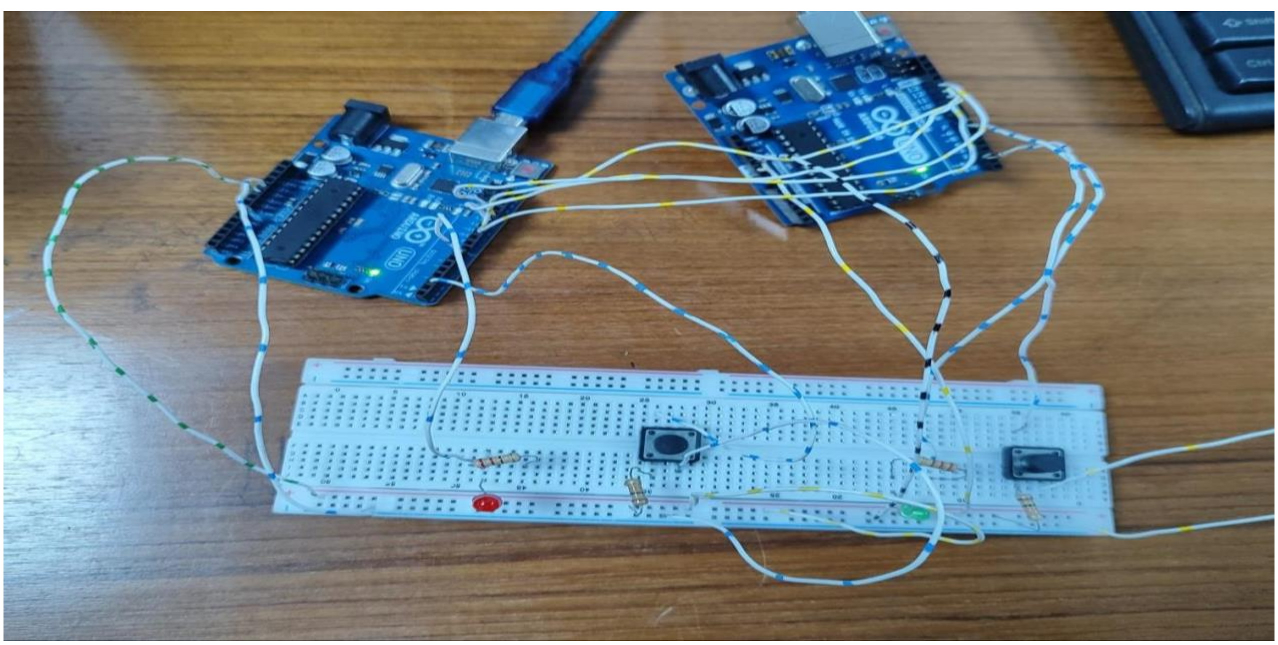

**Figure 2.1: Turn on an LED (Both off).**

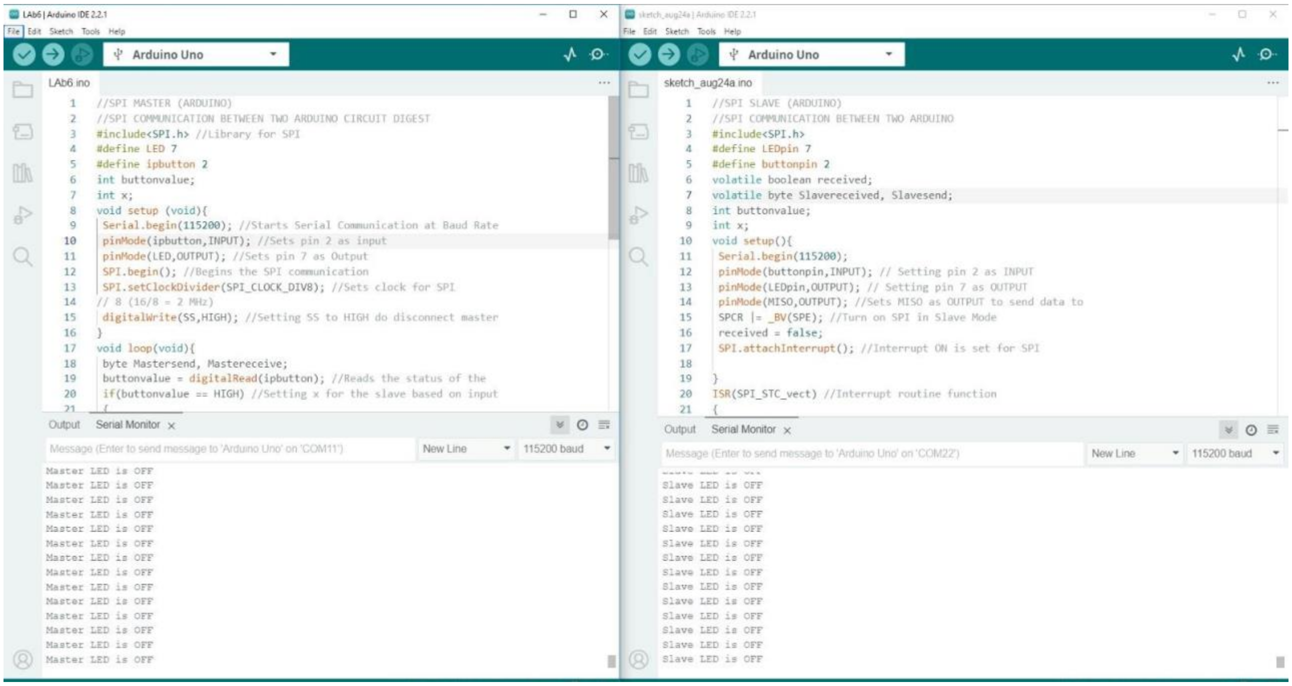


**Figure 2.2: Code for turning on an LED (Both off).**

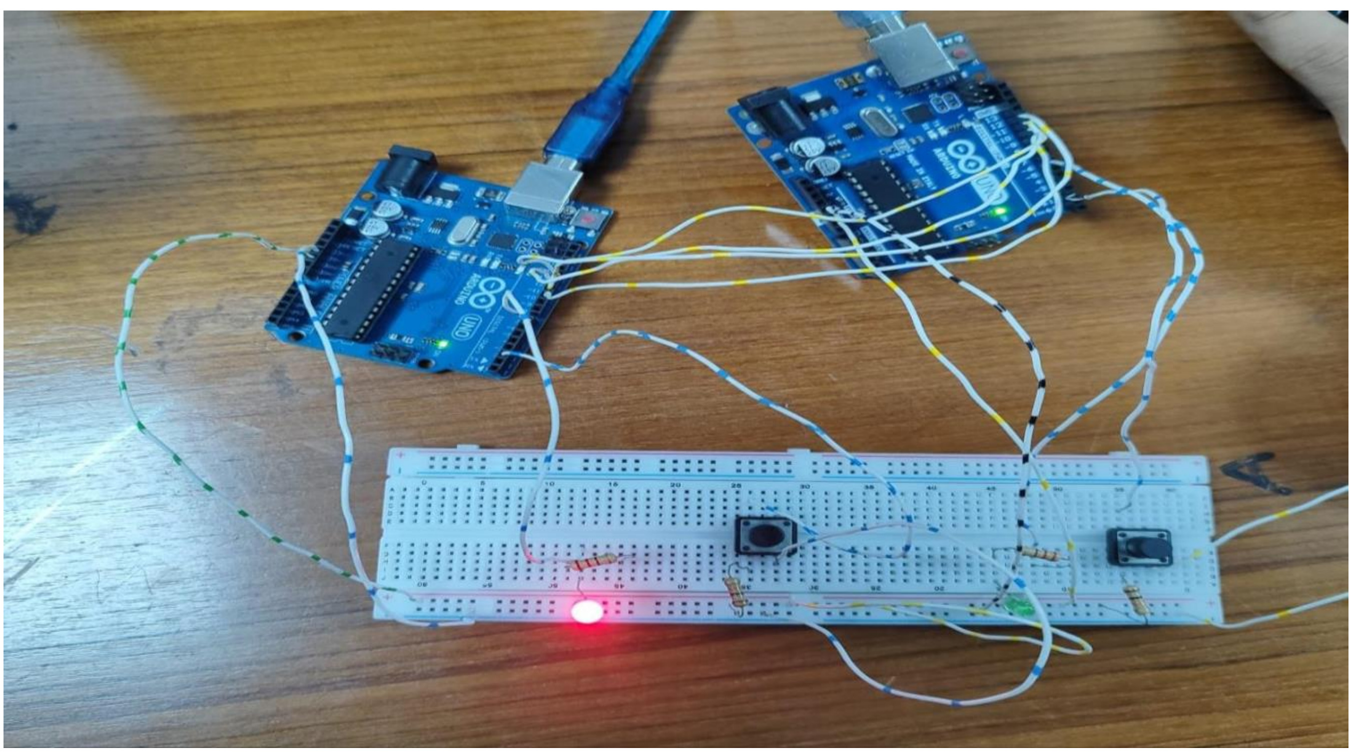

**Figure 2.3: Turn on an LED (Master on).**

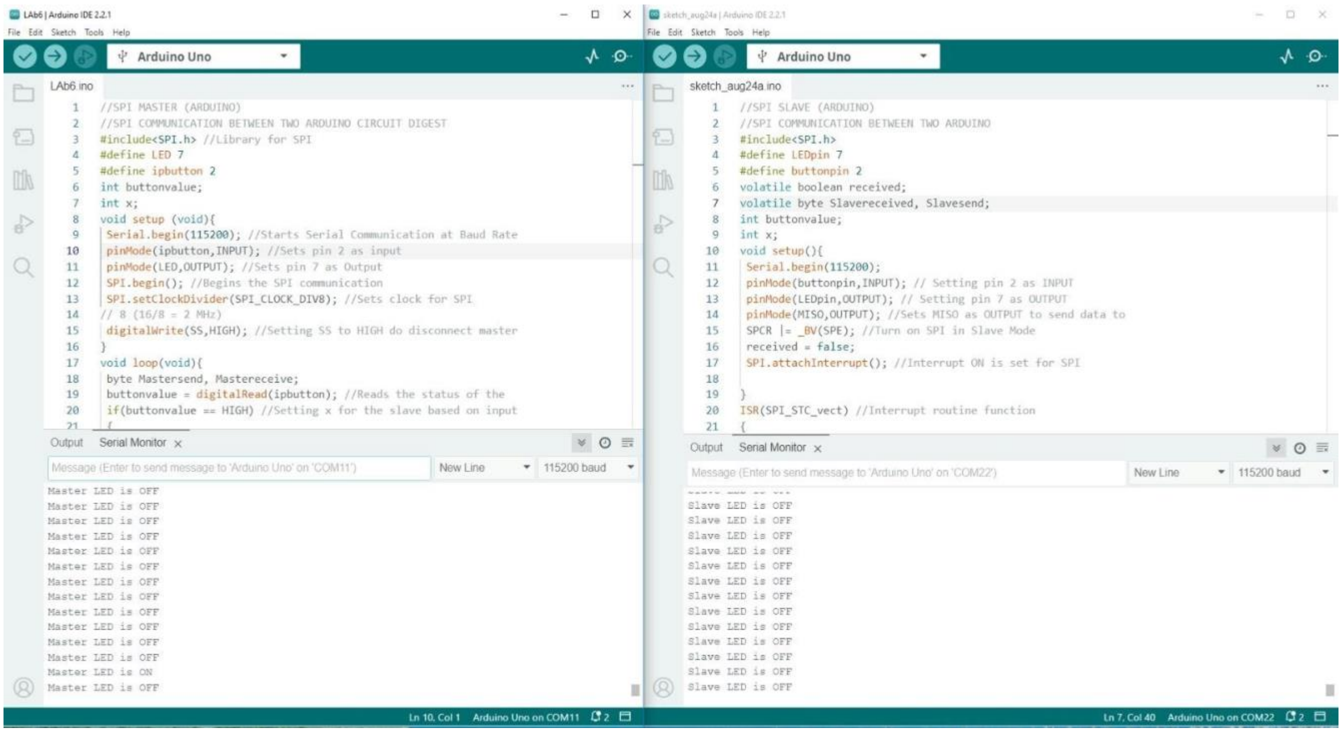


**Figure 2.4: Code for turning on an LED (Master on).**

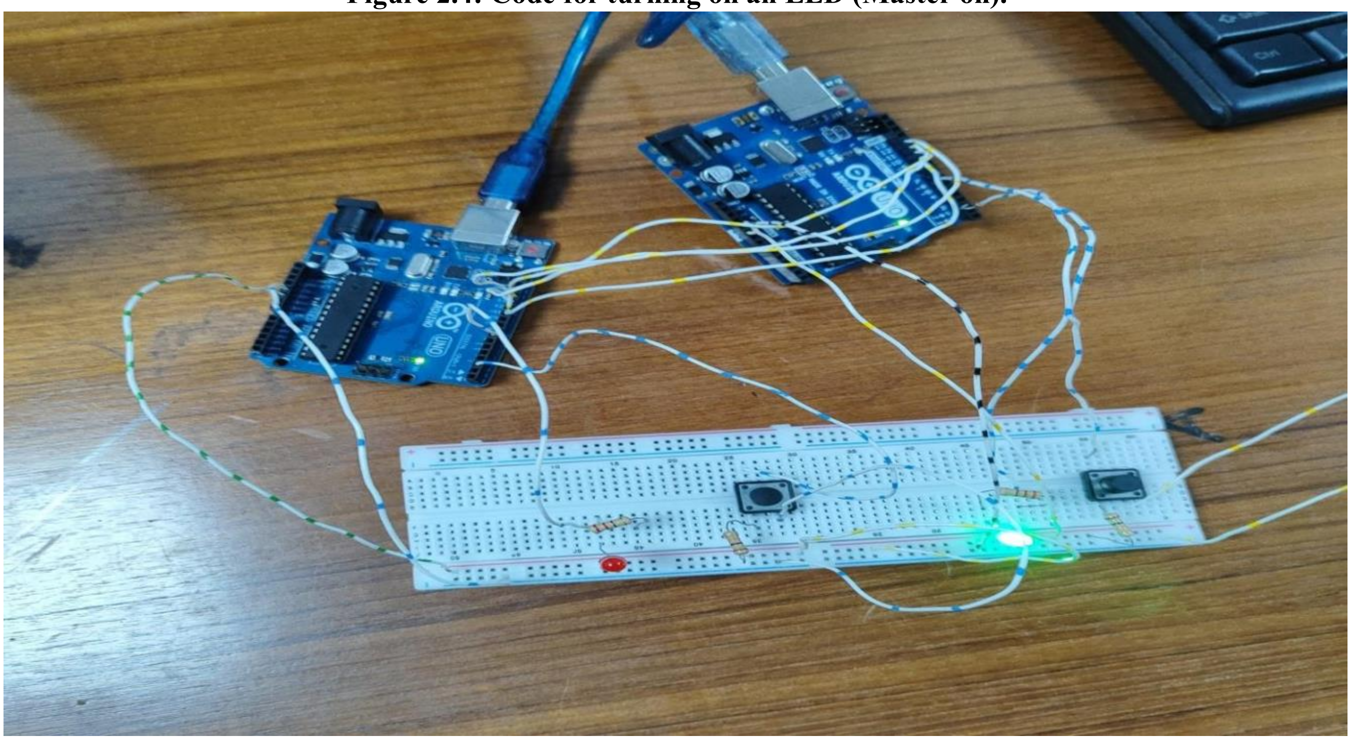

**Figure 2.5: Turn on an LED (Slave on).**

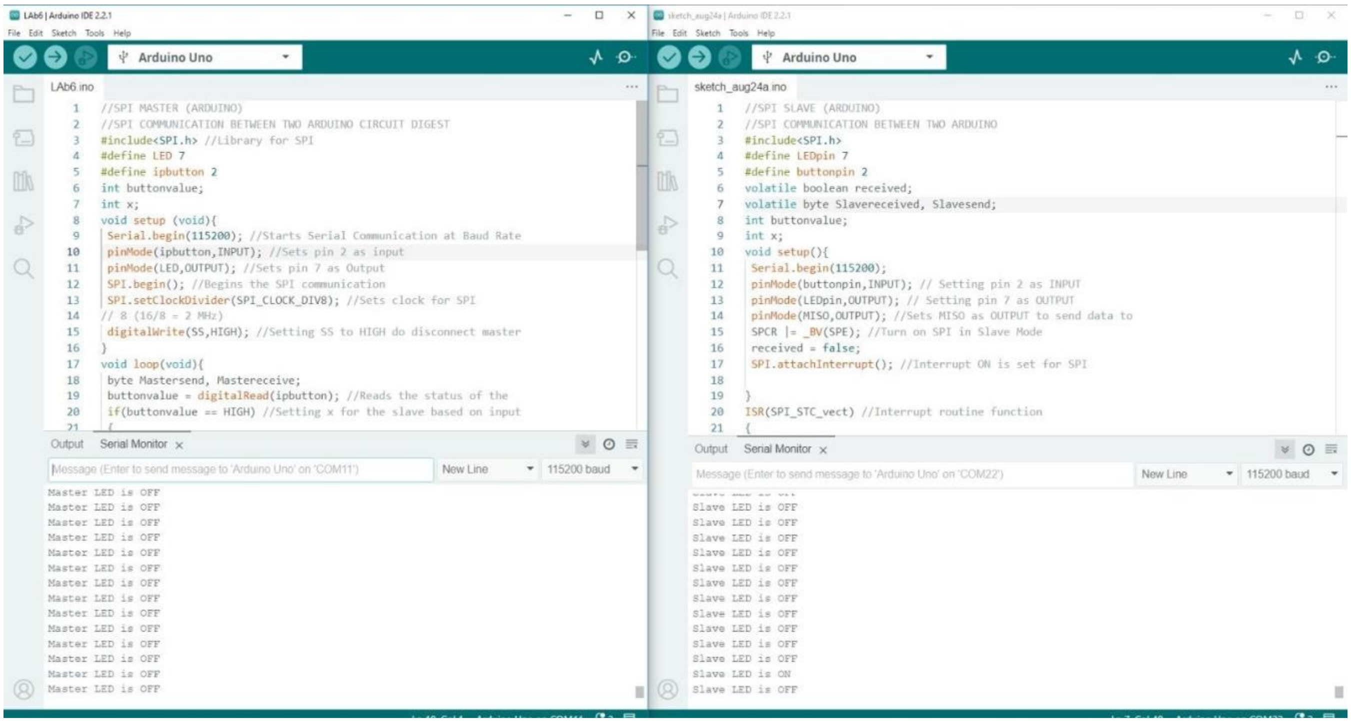


**Figure 2.6: Code for turning on an LED (Slave on).**

## Simulation Output Results:

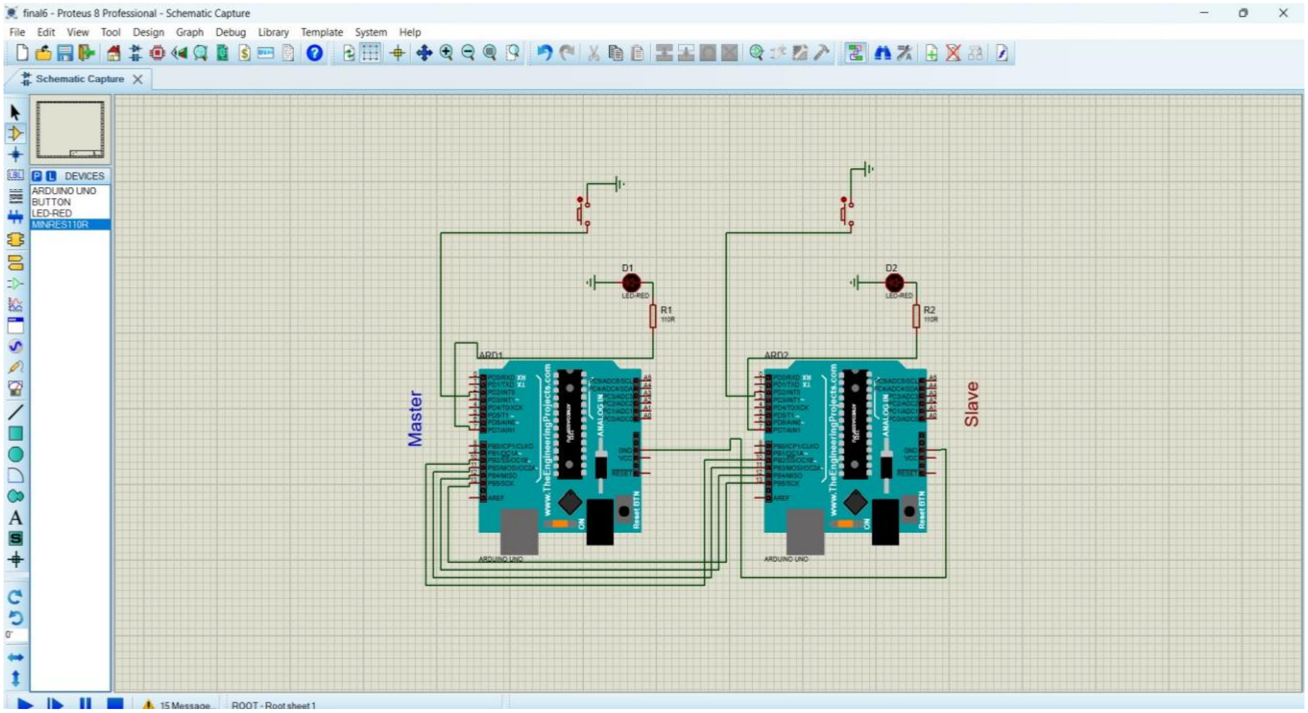


**Figure 2.7: Simulation for turning on an LED (Both off).**

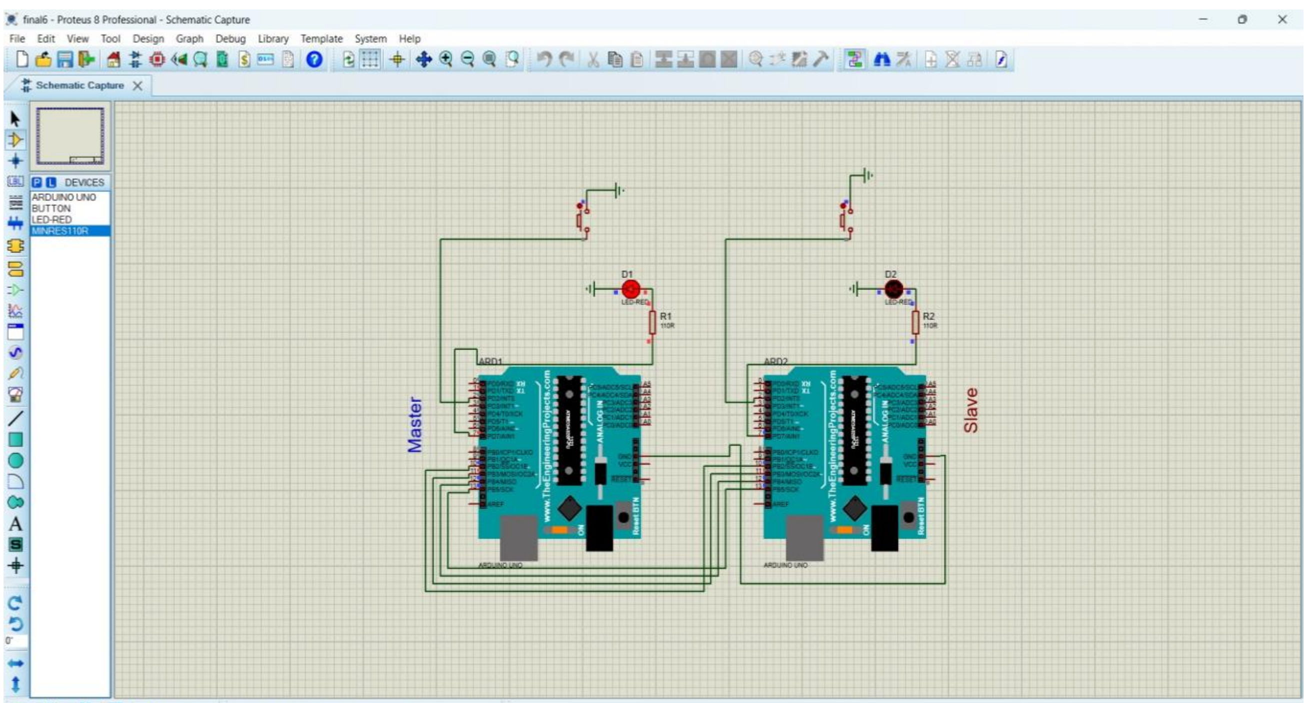


**Figure 2.8: Simulation for turning on an LED (Master on).**

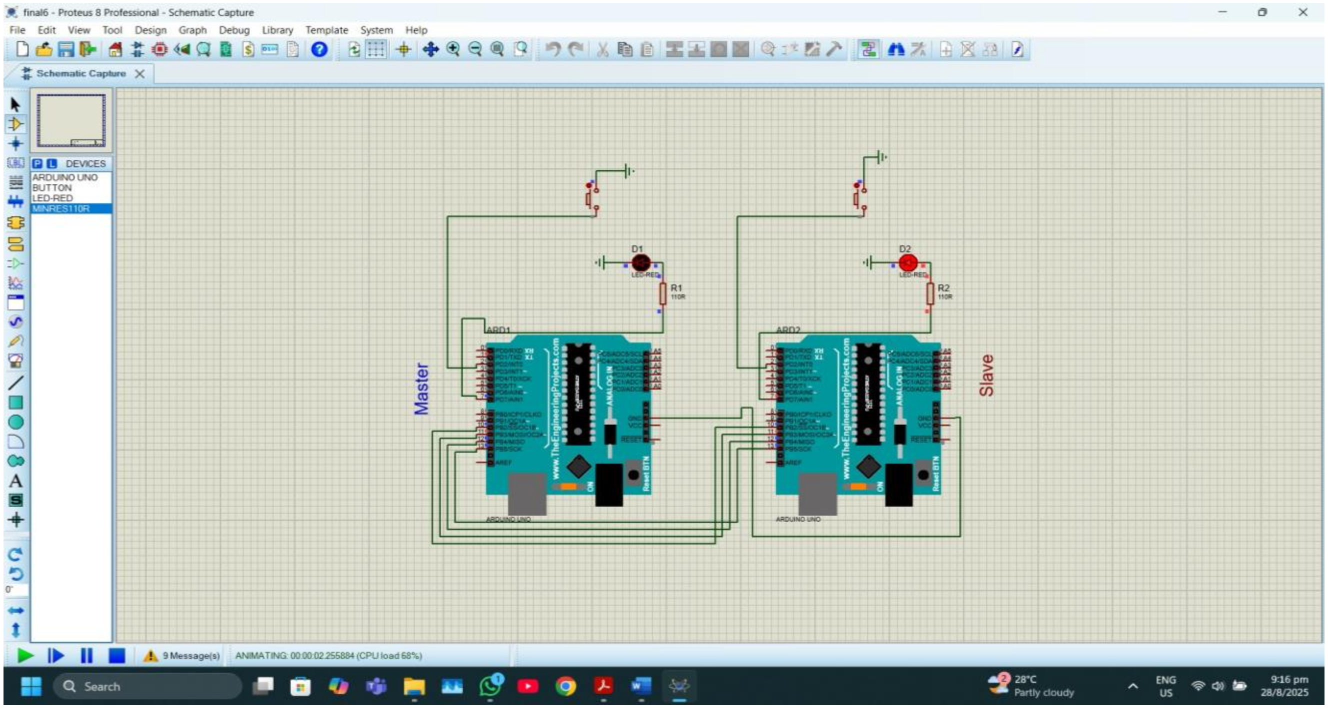


**Figure 2.9: Simulation for turning on an LED (Slave on).**

# Answers to the Questions:

## Question-1 Answer:

a) **Master Code:**

```
//SPI MASTER (ARDUINO)
//SPI COMMUNICATION BETWEEN TWO ARDUINO CIRCUIT DIGEST
#include<SPI.h> //Library for SPI
#define LED 7
#define ipbutton 2
int buttonvalue; int
x;
void setup (void){
Serial.begin(115200); //Starts Serial Communication at Baud Rate 115200 pinMode(ipbutton,INPUT);
//Sets pin 2 as input
pinMode(LED,OUTPUT); //Sets pin 7 as Output

SPI.begin(); //Begins the SPI communication
SPI.setClockDivider(SPI_CLOCK_DIV8); //Sets clock for SPI communication at
// 8 (16/8 = 2 MHz)
digitalWrite(SS,HIGH); //Setting SS to HIGH do disconnect master from slave
} void loop(void){ byte Mastersend, Mastereceive; buttonvalue =
digitalRead(ipbutton); //Reads the status of the pin 2 if(buttonvalue ==
HIGH) //Setting x for the slave based on input at pin 2
{ x =
1; }
else {
x = 0;
}
digitalWrite(SS, LOW); //Starts communication with Slave from the Master
Mastersend = x;
Mastereceive = SPI.transfer(Mastersend); //Sends the Mastersend value to
//the slave and also receives value from the slave if(Mastereceive == 1) //To
set the LED based on the value received from slave
{
digitalWrite(LED,HIGH); //Sets pin 7 HIGH
Serial.println("Master LED is ON");
} else
{
digitalWrite(LED,LOW); //Sets pin 7 LOW
Serial.println("Master LED is OFF");
} delay(1000);
}
```

b) **Slave Code:**

```
//SPI SLAVE (ARDUINO)
//SPI COMMUNICATION BETWEEN TWO ARDUINO
#include<SPI.h>
#define LEDpin 7 #define buttonpin 2 volatile boolean received; volatile byte
Slavereceived, Slavesend; int buttonvalue; int x; void setup(){
Serial.begin(115200); pinMode(buttonpin,INPUT); // Setting pin 2 as INPUT
pinMode(LEDpin,OUTPUT); // Setting pin 7 as OUTPUT
pinMode(MISO,OUTPUT); //Sets MISO as OUTPUT to send data to Master In

SPCR |= _BV(SPE); //Turn on SPI in Slave Mode
received = false;
SPI.attachInterrupt(); //Interrupt ON is set for SPI communication
}
ISR(SPI_STC_vect) //Interrupt routine function
{
Slavereceived = SPDR; // Value received from Master stored in Slavereceived received
= true; //Sets received as True
} void loop() { if(received) //To set LED ON/OFF based on the value
received from Master
{
  if (Slavereceived == 1)
{
digitalWrite(LEDpin, HIGH); //Sets pin 7 as HIGH to turn on LED
Serial.println("Slave LED is ON");
} else
{
digitalWrite(LEDpin,LOW); //Sets pin 7 as LOW to turn off LED
Serial.println("Slave LED is OFF");
} buttonvalue = digitalRead(buttonpin); //Reads the status of the pin
2 if (buttonvalue == HIGH) //To set the value of x to send to Master
{ x =
1; }
else {
x=0;
}
Slave
send
= x;
SPDR = Slavesend; //Sends the x value to the Master via SPDR delay(1000);
}
}
```

**Question-2 Answer:**

Already done in the "Experimental Output Results" section.

## Question-3 Answer:

My ID: **23-50040-1**
Last two digits = **40** So:

- **Input Port = 4**
- **Output Port = 0** That means:
- Button (input) should be connected to **pin 4**
- LED (output) should be connected to **pin 0**

## Code:

### Master:

```
//SPI MASTER (ARDUINO)
//SPI COMMUNICATION BETWEEN TWO ARDUINOS
#include <SPI.h>

#define LED 0          // Output pin based on ID
#define ipbutton 4     // Input pin based on ID

int buttonvalue;
int x;

void setup(void) {
  Serial.begin(115200);         // Starts Serial Communication
pinMode(ipbutton, INPUT);      // Sets pin 4 as input
  pinMode(LED, OUTPUT);         // Sets pin 0 as output

  SPI.begin();                  // Begins the SPI communication
  SPI.setClockDivider(SPI_CLOCK_DIV8);  // 2 MHz SPI speed
digitalWrite(SS, HIGH);        // Disconnect Master from Slave
}

void loop(void) {   byte
Mastersend, Mastereceive;
  buttonvalue = digitalRead(ipbutton); // Reads the status of pin 4   if(buttonvalue == HIGH) {
    x = 1;
} else {
x = 0;
  }

  digitalWrite(SS, LOW);              // Start communication
  Mastersend = x;
  Mastereceive = SPI.transfer(Mastersend); // Exchange data with slave
```

```
  if(Mastereceive == 1) {     digitalWrite(LED,
HIGH);          // LED ON
    Serial.println("Master LED is ON");
  } else {
    digitalWrite(LED, LOW);           // LED OFF
    Serial.println("Master LED is OFF");
  }
  delay(1000);
}
```

**Slave:**

```
//SPI SLAVE (ARDUINO)
//SPI COMMUNICATION BETWEEN TWO ARDUINOS
#include <SPI.h>

#define LEDpin 0       // Output pin based on ID
#define buttonpin 4    // Input pin based on ID

volatile boolean received; volatile byte
Slavereceived, Slavesend; int
buttonvalue; int x;

void setup() {
Serial.begin(115200);
  pinMode(buttonpin, INPUT);    // Pin 4 as input
pinMode(LEDpin, OUTPUT);      // Pin 0 as output
  pinMode(MISO, OUTPUT);       // Set MISO as OUTPUT

  SPCR |= _BV(SPE);            // Enable SPI in Slave Mode
  received = false;
  SPI.attachInterrupt();       // Enable SPI interrupt
}

ISR(SPI_STC_vect) {
  Slavereceived = SPDR;        // Receive from Master
  received = true;
}
v
o
i
d
l
o
o
p
()
```

```
{
if
(r
e
c
ei
v
e
d
)
{
if
(
S
la
v
e
r
e
c
ei
v
e
d
=
=
1
)
{
      digitalWrite(LEDpin, HIGH);   // LED ON
      Serial.println("Slave LED is ON");
    } else {
      digitalWrite(LEDpin, LOW);    // LED OFF
      Serial.println("Slave LED is OFF");
    }

    buttonvalue = digitalRead(buttonpin);  // Read button state
if (buttonvalue == HIGH) {
      x = 1;
} else {
x = 0;
    }

    Slavesend = x;
    SPDR = Slavesend;   // Send data to Master
    delay(1000);
  }
}
```

**Result (input 4, Output 0):**

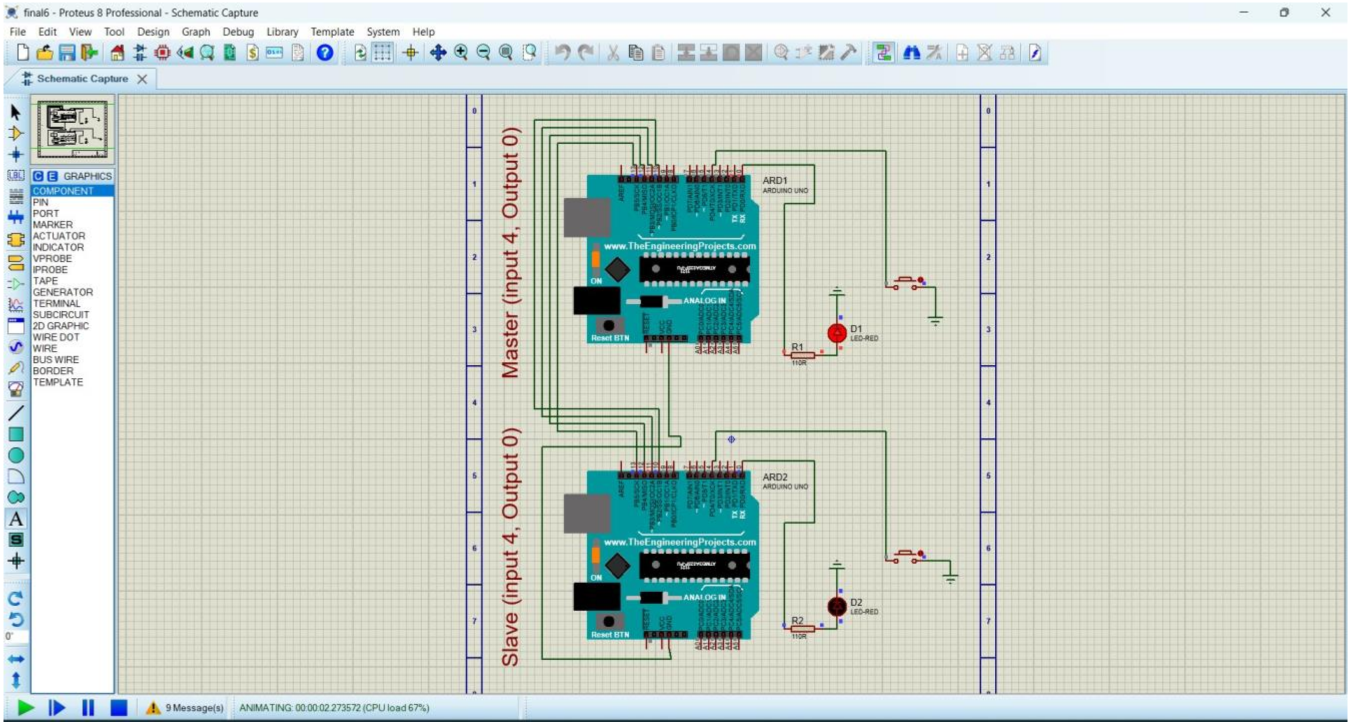


Figure 2.10: Simulation for turning on an LED (Master on).

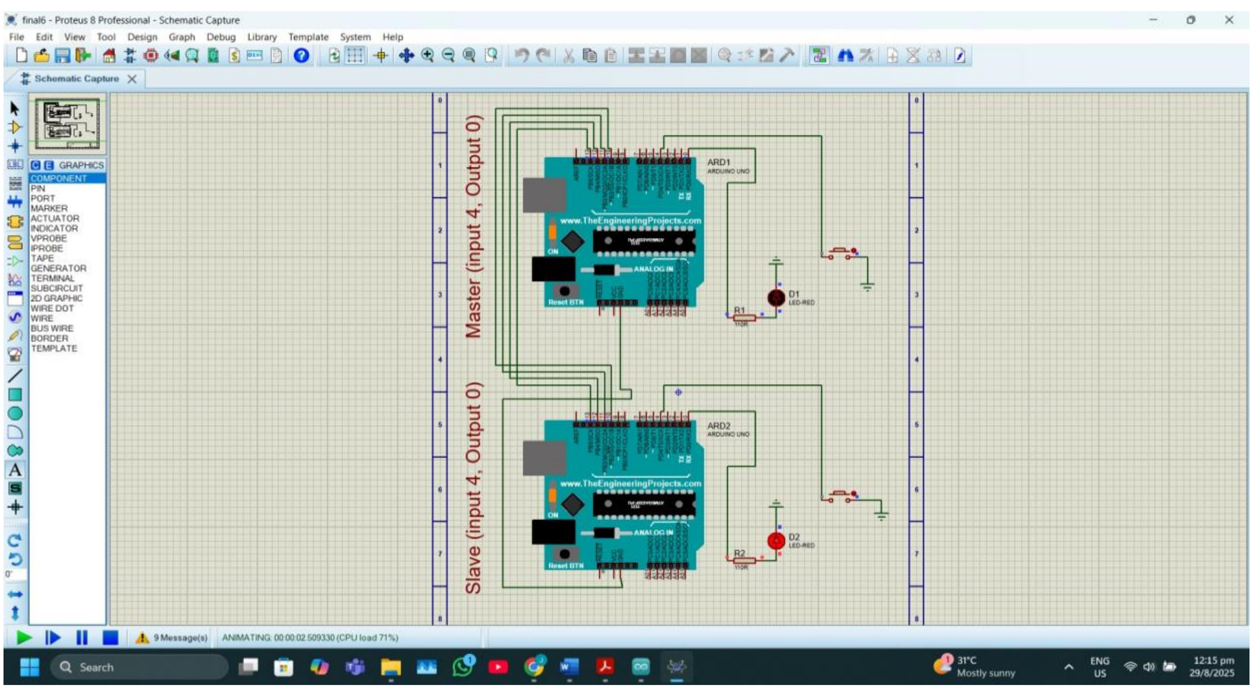


Figure 2.11: Simulation for turning on an LED (Slave on).

**Question-4 Answer:**

Based on the pin configuration from Question 3 (input at pin 4 and output at pin 0), the LED blink program and LED control system were simulated in Proteus.

**Simulation Methodology**

- Based on the pin configuration from Question 3 (input at pin 4 and output at pin 0).
- In Proteus, an Arduino UNO was placed with:
  - LED connected to pin 0 through a resistor.
  - Push button connected to pin 4 with a pull-down resistor.
- Arduino code was compiled in the IDE, and the HEX file was uploaded to the Proteus Arduino.
- On running the simulation:
  - In the blink program → LED blinked at regular intervals. o In the control program → LED responded to button presses.

## Discussion and Real-Life Scenario:

The experiment successfully demonstrated SPI communication between two Arduino boards, as well as LED blinking and control using push buttons in Proteus simulation. Configuring the ports based on the last two digits of the student ID ensured a personalized setup, making the task more engaging and unique. The results showed that the master and slave boards could exchange data reliably, and that input–output operations (button press and LED response) were correctly implemented both in hardware and simulation.

Such experiments reflect real-world applications of microcontroller communication and input–output systems. For example, in home automation, a master controller can gather input from sensors (like motion detectors or switches) and control devices such as lights or fans. Similarly, in industrial automation, machines often communicate via SPI or similar protocols, where one module reads button or sensor inputs and another module controls motors, indicators, or alarms. The simple LED–button setup is a scaled-down version of these systems, showing how user input can control an output device through programmed logic.

## References:

[1] J. G. Kassakian, J. M. Miller, and A. Emadi, “Integrated Control of Power Electronics and Embedded Systems in Smart Traffic Applications,” *IEEE Transactions on Power Electronics*, vol. 35, no. 6, pp. 6187–6200, Jun. 2020.

[2] S. Srivastava, A. Goel, and S. Bansal, “Low-Power Event Driven Microcontroller Systems for IoT Applications,” *IEEE Internet of Things Journal*, vol. 8, no. 8, pp. 6652–6663, Apr. 2021.